



JavaScript Fundamentals

Functions, Objects, DOM & Async Patterns

Chapter Overview

- 01** Functions in Depth
- 02** Objects and Classes
- 03** Modern JavaScript
- 04** The DOM and Event Handling
- 05** Error Handling and Debugging
- 06** Asynchronous JavaScript



Functions in Depth

Section 1



Why Functions Matter

Code Reuse

- Write once, call anywhere — eliminates copy-paste

Abstraction

- Hide complexity behind a meaningful name

Composition

- Build complex behavior from small, tested pieces

First-Class Citizens

- In JavaScript, functions are values — assign them to variables, pass as arguments, return from other functions



Three Ways to Define Functions

Three Ways to Define Functions

Function Declaration

```
function greet(name) {  
  return `Hello, ${name}!`;  
}
```

- > Hoisted — can call before definition
- > Has its own this binding
- > Named — appears in stack traces

Function Expression

```
const greet = function(name) {  
  return `Hello, ${name}!`;  
};
```

- > NOT hoisted — must define before use
- > Has its own this binding
- > Can be anonymous or named

Arrow Function

```
const greet = (name) => {  
  return `Hello, ${name}!`;  
};  
  
// Short form  
const greet = name => `Hello,  
${name}!`;
```

- > NOT hoisted
- > Inherits this from parent scope
- > Concise syntax — ideal for callbacks

Use arrow functions for callbacks · Use declarations for top-level functions

Function Declarations vs Expressions

Function Declarations

- Hoisted — available before the line they appear on
- Always named — the name appears in error stack traces
- Good for top-level utility functions

Function Expressions

- NOT hoisted — must be defined before use
- Assigned to a variable (const or let)
- Can be anonymous or named



Declarations vs Expressions

```
// Declaration – hoisted, can call before definition  
greet("Alice"); // Works!
```

```
function greet(name) {  
  return `Hello, ${name}!`;  
}
```

```
// Expression – NOT hoisted  
const farewell = function(name) {  
  return `Goodbye, ${name}!`;  
};
```

```
farewell("Bob"); // Must come after definition
```

Arrow Functions

Concise Syntax

- Shorter than function expressions — ideal for callbacks
- Single expression? Omit braces and return keyword
- Single parameter? Omit parentheses

Key Difference: this

- Arrow functions inherit this from the surrounding scope
- Regular functions create their own this binding

When to Use Each

- Arrows for callbacks, array methods, short inline logic
- Declarations for top-level named functions



Arrow Function Syntax

```
// Standard arrow
const add = (a, b) => {
  return a + b;
};

// Concise body – implicit return
const add = (a, b) => a + b;

// Single parameter – no parens needed
const double = n => n * 2;

// In practice: array methods
const names = ['Alice', 'Bob', 'Charlie'];
const upper = names.map(n => n.toUpperCase());
const long = names.filter(n => n.length > 3);
```

Parameters and Default Values

Default Parameters

- Provide fallback values when arguments are omitted
- Syntax: `function greet(name = 'World')`

Rest Parameters (...args)

- Collects remaining arguments into a real array
- Must be the last parameter in the list
- Replaces the old arguments object



Default and Rest Parameters

```
// Default parameters
function greet(name = "World") {
  return `Hello, ${name}!`;
}
greet();           // "Hello, World!"
greet("Alice");    // "Hello, Alice!"

// Rest parameters – collect into an array
function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}
sum(1, 2, 3);      // 6
sum(10, 20, 30, 40); // 100
```

Higher-Order Functions and Closures

Higher-Order Functions

- A function that takes a function as an argument or returns one
- Array methods like map, filter, reduce are higher-order
- Enables powerful patterns: callbacks, event handlers, middleware
- Core to functional programming in JavaScript

Closures

- A function that remembers variables from its outer scope
- The inner function 'closes over' the outer variables
- Used for data privacy, factory functions, and memoization



Closures in Action

```
// Closure: inner function remembers outer variable
function createCounter() {
  let count = 0; // private – only accessible via returned fn
  return {
    increment() { return ++count; },
    getCount() { return count; },
  };
}

const counter = createCounter();
counter.increment(); // 1
counter.increment(); // 2
counter.getCount();  // 2
// count is not directly accessible from outside
```

Objects and Classes

Section 2



Anatomy of a JavaScript Object

Anatomy of a JavaScript Object

```
const account = {}
```

Property	Value	Type
name	"John Doe"	String
balance	5000	Number
active	true	Boolean
deposit	function()	Method

```
account.name  
=> "John Doe"
```

```
account["balance"]  
=> 5000
```

```
account.deposit(100)  
=> balance is now 5100
```

Object Fundamentals

Creating Objects

- Object literal: `const user = { name: 'Alice', age: 28 }`
- Properties are key-value pairs; values can be any type

Accessing Properties

- Dot notation: `user.name` — clean, preferred for known keys
- Bracket notation: `user['name']` — required for dynamic keys

Methods

- Functions stored as object properties
- Use shorthand syntax: `greet() { }` instead of `greet: function() { }`



Working with Objects

```
const product = {
  name: "Widget",
  price: 29.99,
  inStock: true,
  tags: ['sale', 'new'],
  display() {
    return `${this.name}: ${this.price}`;
  },
};

product.name;           // "Widget"
product["price"];       // 29.99
product.display();      // "Widget: $29.99"
product.rating = 4.5;    // Add new property
delete product.inStock;  // Remove property
```

The this Keyword

What is this?

- A reference to the object that is executing the current function
- Its value depends on HOW the function is called, not where defined

this in Different Contexts

- Object method: this refers to the object (user.greet())
- Standalone function: this is undefined in strict mode
- Arrow function: this is inherited from the enclosing scope

Best Practice

- Use arrow functions in callbacks to avoid losing this context



Iterating Over Objects

for...in Loop

- Iterates over an object's enumerable property names (keys)
- Use `hasOwnProperty()` to skip inherited properties

Object Static Methods

- `Object.keys(obj)` — returns an array of key names
- `Object.values(obj)` — returns an array of values
- `Object.entries(obj)` — returns `[[key, value], ...]` pairs



Iterating Over Objects

```
const settings = { theme: "dark", fontSize: 14, lang: "en" };

// Object.keys → array of keys
Object.keys(settings);    // ["theme", "fontSize", "lang"]

// Object.entries → key-value pairs
for (const [key, value] of Object.entries(settings)) {
  console.log(`${key}: ${value}`);
}

// Spread to merge/copy
const updated = { ...settings, fontSize: 16 };
// { theme: "dark", fontSize: 16, lang: "en" }
```

Classes: Structured Object Creation

What are Classes?

- Syntactic sugar over JavaScript's prototype-based inheritance
- A blueprint for creating objects with shared behavior

Key Parts

- `constructor()` — initializes new instances
- `Methods` — shared functions available on every instance
- `extends` — create a subclass that inherits from a parent

Why Use Classes?

- Clear structure for object-oriented patterns
- Foundation for React class components and TypeScript



Class Syntax and Inheritance

```
class Animal {
  constructor(name, sound) {
    this.name = name;
    this.sound = sound;
  }
  speak() {
    return `${this.name} says ${this.sound}`;
  }
}

class Dog extends Animal {
  constructor(name) { super(name, "Woof"); }
  fetch(item) { return `${this.name} fetches ${item}`; }
}

const rex = new Dog("Rex");
rex.speak();           // "Rex says Woof"
rex.fetch("ball");    // "Rex fetches ball"
```

Modern JavaScript

Section 3



Destructuring & Spread/Rest Operators

Destructuring & Spread/Rest Operators

Object Destructuring

```
const user = { name: "Alice", age: 28 };

// Extract properties
const { name, age } = user;

// Rename + default
const { name: n, role = "user" } = user;
```

Spread Operator (...)

```
// Copy & merge arrays
const copy = [...colors];
const all = [...a, ...b];

// Copy & merge objects
const updated = { ...user, age: 29 };
```

Array Destructuring

```
const colors = ["red", "green", "blue"];

const [first, second] = colors;
// first = "red", second = "green"

const [, , third] = colors;
// Skip elements: third = "blue"
```

Rest Parameters (...)

```
// Collect remaining args
function sum(...nums) {
  return nums.reduce((a, b) => a + b);
}

// Rest in destructuring
const { name, ...rest } = user;
```



Object Destructuring

What is Destructuring?

- Extract properties from objects into standalone variables
- Replaces repetitive dot-notation access

Renaming and Defaults

- Rename: `const { name: userName } = user`
- Default: `const { role = 'member' } = user`

In Function Parameters

- Destructure directly in the parameter list for clean APIs
- `function greet({ name, age }) { ... }`



Destructuring in Practice

```
const user = { name: "Alice", age: 28, role: "admin" };

// Object destructuring
const { name, age } = user;

// Rename + default value
const { name: userName, team = "unassigned" } = user;

// Array destructuring
const [first, , third] = ["a", "b", "c"];
// first = "a", third = "c"

// In function parameters
function greet({ name, role }) {
  return `${name} (${role})`;
}
greet(user); // "Alice (admin)"
```



Spread and Rest Operators

Spread (...) — Expands

- Copy arrays: `const copy = [...original]`
- Merge arrays: `const all = [...arr1, ...arr2]`
- Copy/merge objects: `const updated = { ...obj, key: newVal }`

Rest (...) — Collects

- Gather remaining function arguments: `function fn(...args)`
- Gather remaining properties: `const { id, ...rest } = obj`
- Same syntax (...), different meaning based on context



Optional Chaining and Nullish Coalescing

Optional Chaining (?.)

- Safely access deeply nested properties without crashing
- `user?.address?.city` returns undefined instead of throwing
- Works with methods: `user?.getName?.()` and arrays: `arr?.[0]`

Nullish Coalescing (??)

- Provides a fallback only for null or undefined
- Unlike `||`, it does NOT treat 0, "", or false as missing
- `const port = config.port ?? 3000`



Modern Operators in Practice

```
const user = {  
  name: "Alice",  
  address: { city: "Boston" },  
  settings: { notifications: 0 },  
};  
  
// Optional chaining – safe deep access  
user?.address?.city;      // "Boston"  
user?.phone?.number;      // undefined (no error)  
  
// Nullish coalescing – fallback for null/undefined only  
user?.age ?? 25;           // 25 (age is undefined)  
user?.settings?.notifications ?? true; // 0 (not null!)  
  
// Contrast with || (treats 0 as falsy)  
user?.settings?.notifications || true; // true (wrong!)
```



Modules: import and export

Why Modules?

- Split code into separate files with clear responsibilities
- Each module has its own scope — no global variable pollution

Named Exports

- Export multiple values: `export function add() { }`
- Import by name: `import { add, subtract } from './math.js'`

Default Exports

- One default per file: `export default class App { }`
- Import with any name: `import App from './App.js'`



ES Modules in Practice

```
// math.js – named exports
export function add(a, b) { return a + b; }
export function subtract(a, b) { return a - b; }
export const PI = 3.14159;
```

```
// app.js – named imports
import { add, subtract, PI } from './math.js';
console.log(add(2, 3)); // 5
```

```
// utils.js – default export
export default function formatDate(d) {
  return d.toLocaleDateString();
}
```

```
// app.js – default import (any name)
import fmt from './utils.js';
```



The DOM and Event Handling

Section 4



The Document Object Model (DOM)

The Document Object Model (DOM)

The DOM Tree

```
document
├── html
│   ├── head
│   │   ├── title
│   │   └── meta
│   └── body
│       ├── header
│       │   └── h1
│       ├── main
│       │   ├── p
│       │   └── ul
│       │       ├── li
│       │       └── li
│       └── footer
```

Every HTML element becomes a node in the tree. JavaScript can read, add, remove, and modify any node.

Selecting Elements

```
getElementById("id")
querySelector(".class")
querySelectorAll("tag")
```

Modifying Elements

```
el.textContent = "text"
el.innerHTML = "<b>html</b>"
el.style.color = "red"
el.classList.add("active")
```

Creating & Removing

```
document.createElement("div")
parent.appendChild(child)
el.remove()
```

Selecting DOM Elements

Single Element Selectors

- `getElementById('id')` — fastest, returns one element
- `querySelector('.class')` — CSS selector, returns first match

Multiple Element Selectors

- `querySelectorAll('selector')` — returns a `NodeList` of all matches
- Use `forEach` or `spread` to iterate: `[...nodeList].map(...)`

Best Practice

- Prefer `querySelector/querySelectorAll` — consistent CSS selector syntax



Modifying DOM Elements

Content

- `textContent` — get/set text only (safe, no HTML parsing)
- `innerHTML` — get/set HTML (use cautiously, XSS risk)

Attributes and Styles

- `setAttribute('name', 'value') / getAttribute('name')`
- `element.style.property = 'value'` for inline styles
- `classList.add(), .remove(), .toggle()` for CSS classes

Creating and Removing

- `document.createElement('tag') → parent.appendChild(child)`
- `element.remove()` — removes element from the DOM



DOM Manipulation Examples

```
// Select elements
const heading = document.querySelector("h1");
const items = document.querySelectorAll(".item");

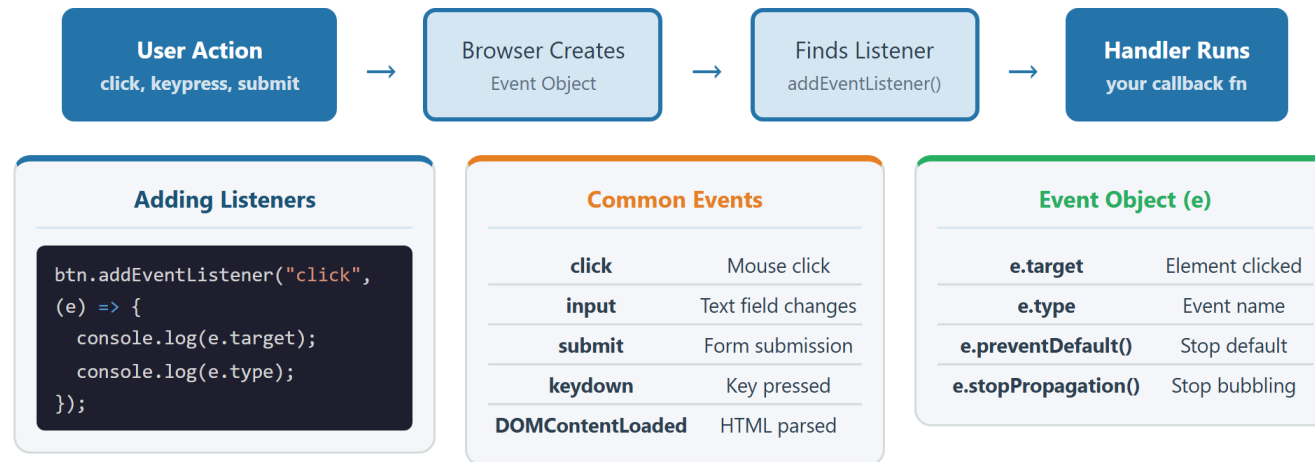
// Modify content and style
heading.textContent = "Updated Title";
heading.classList.add("highlight");

// Create and append a new element
const li = document.createElement("li");
li.textContent = "New item";
document.querySelector("ul").appendChild(li);

// Remove an element
document.querySelector(".old").remove();
```

Event Handling in JavaScript

Event Handling in JavaScript



addEventListener and the Event Object

Adding Event Listeners

- `element.addEventListener('event', handlerFn)` — the modern approach
- Multiple listeners per element, easy to remove later

The Event Object (e)

- Passed automatically to the handler function
- `e.target` — the element that triggered the event
- `e.preventDefault()` — stop default behavior (form submit, link)
- `e.stopPropagation()` — stop event from bubbling up



Event Handling in Practice

```
// Click handler
const btn = document.querySelector("#submit-btn");
btn.addEventListener("click", (e) => {
  console.log('Clicked!', e.target);
});

// Form submission – prevent page reload
const form = document.querySelector("form");
form.addEventListener("submit", (e) => {
  e.preventDefault();
  const name = form.querySelector("#name").value;
  console.log('Submitted:', name);
});

// Keyboard event
document.addEventListener("keydown", (e) => {
  if (e.key === "Escape") closeModal();
});
```



Event Delegation

The Problem

- Adding listeners to many elements is slow and wasteful
- Dynamically added elements won't have listeners

The Solution: Event Delegation

- Add ONE listener to a parent element
- Use `e.target` to determine which child was clicked
- Works because events bubble up from child to parent



Event Delegation Example

```
// Instead of adding a listener to EVERY <li>...  
// Add ONE listener to the parent <ul>  
const list = document.querySelector("#todo-list");  
  
list.addEventListener("click", (e) => {  
  // Check if the clicked element is an <li>  
  if (e.target.tagName === "LI") {  
    e.target.classList.toggle("done");  
  }  
});  
  
// This works even for items added AFTER the listener!  
const newItem = document.createElement("li");  
newItem.textContent = "New task";  
list.appendChild(newItem); // Click handler works
```



Error Handling and Debugging

Section 5



Error Handling & Debugging

Error Handling & Debugging

try / catch / finally

```
try {  
  // Code that might fail  
  const data = JSON.parse(input);  
} catch (error) {  
  // Handle the error  
  console.error(error.message);  
} finally {  
  // Always runs  
  cleanup();  
}
```

Throwing Custom Errors

```
function divide(a, b) {  
  if (b === 0) {  
    throw new Error("Cannot divide by zero");  
  }  
  return a / b;  
}
```

Common Error Types

SyntaxError	Invalid code
ReferenceError	Undefined variable
TypeError	Wrong type operation
RangeError	Value out of range
URIError	Bad URI encoding

Debugging Tips

- > **console.log()** — inspect values
- > **console.table()** — view arrays/objects
- > **debugger;** — pause in DevTools
- > **Stack trace** — read bottom to top

try / catch / finally

try Block

- Wrap code that might throw an error
- If an error occurs, execution jumps to catch

catch Block

- Receives the error object with message and stack properties
- Handle the error: log it, show a message, retry, or re-throw

finally Block

- Runs regardless of success or failure — cleanup code goes here



try / catch / finally

```
function parseConfig(jsonString) {  
  try {  
    const config = JSON.parse(jsonString);  
    return config;  
  } catch (error) {  
    console.error("Invalid JSON:", error.message);  
    return {}; // safe fallback  
  } finally {  
    console.log("Parse attempt complete");  
  }  
}  
  
parseConfig('{"port": 3000}'); // { port: 3000 }  
parseConfig("not json");      // {} + error logged
```

Throwing Custom Errors

Why Throw?

- Validate inputs and fail fast with a clear message
- Callers can catch the error and decide what to do

throw new Error('message')

- Creates an Error object with a stack trace
- Custom error types: `class ValidationError extends Error { }`

Debugging with DevTools

- `console.log`, `console.table`, `console.group` for inspection
- `debugger`; statement pauses execution in DevTools
- Read stack traces bottom-to-top to find the root cause

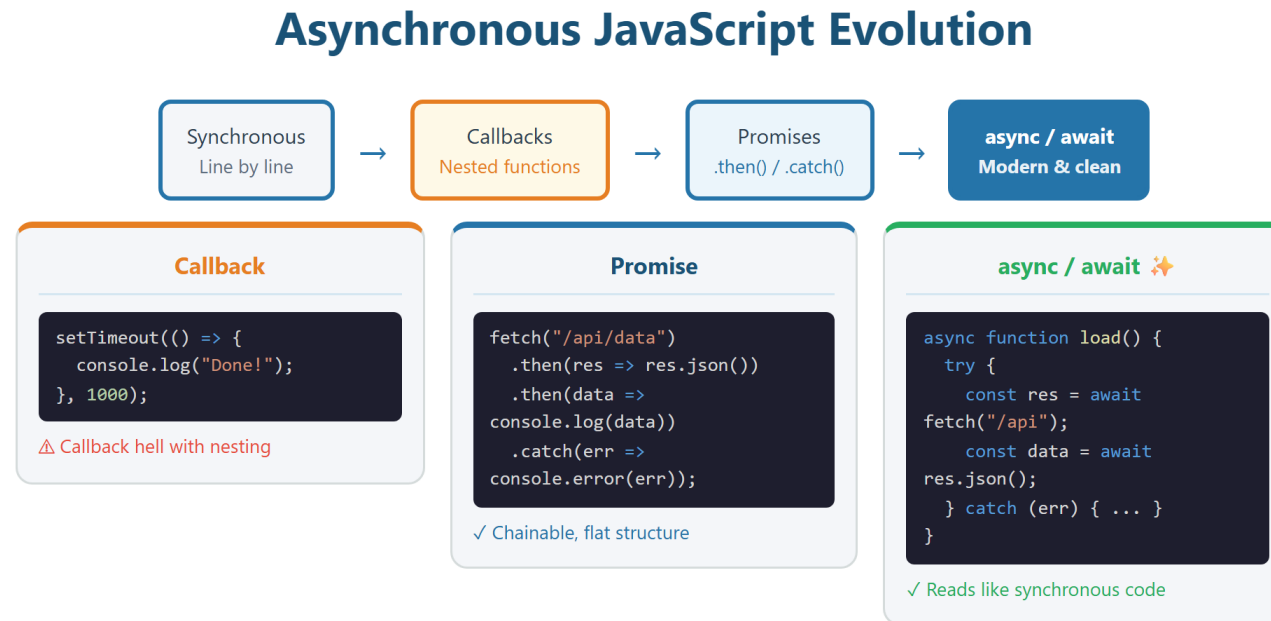


Asynchronous JavaScript

Section 6



Asynchronous JavaScript Evolution



Why Async? The Single-Threaded Problem

JavaScript is Single-Threaded

- Only one thing runs at a time on the main thread
- Long tasks (network requests, file I/O) would freeze the UI

The Event Loop Solution

- Offload slow tasks to the browser/Node.js runtime
- When the task completes, a callback is queued
- The event loop picks up callbacks when the main thread is free



Callbacks and Their Limitations

What is a Callback?

- A function passed to another function, called when work completes
- `setTimeout`, event listeners, and Node.js `fs.readFile` all use callbacks

Callback Hell

- Nested callbacks create deeply indented, hard-to-read code
- Error handling becomes repetitive and fragile
- Promises solve both these problems



Promises

What is a Promise?

- An object representing a value that may be available in the future
- Three states: pending → fulfilled (resolved) or rejected

Chaining with `.then()` and `.catch()`

- `.then(fn)` runs when the promise resolves successfully
- `.catch(fn)` runs when the promise rejects (an error occurred)
- `.finally(fn)` runs after resolve or reject — cleanup code
- Chaining: `.then().then().catch()` — flat, readable structure



Promises and the Fetch API

```
// fetch returns a Promise
fetch("https://api.example.com/users")
  .then(response => {
    if (!response.ok) throw new Error('HTTP ' + response.status);
    return response.json();
  })
  .then(users => {
    console.log('Users:', users);
  })
  .catch(error => {
    console.error('Failed:', error.message);
  });
```



async / await — Modern Async Syntax

What is async/await?

- Syntactic sugar over Promises — same behavior, cleaner code
- async marks a function as returning a Promise
- await pauses execution until the Promise resolves

Error Handling with try/catch

- Wrap await calls in try/catch — just like synchronous error handling
- Reads like synchronous code but runs asynchronously



async / await in Practice

```
async function loadUsers() {
  try {
    const response = await fetch("/api/users");
    if (!response.ok) {
      throw new Error(`HTTP ${response.status}`);
    }
    const users = await response.json();
    console.log("Loaded", users.length, "users");
    return users;
  } catch (error) {
    console.error("Failed to load users:", error.message);
    return [];
  }
}

// Call it
const users = await loadUsers();
```



Key Takeaways

Key Takeaways

1. Functions are first-class citizens — use arrow functions for callbacks, declarations for top-level
2. Objects are key-value collections — use classes for structured creation with shared behavior
3. Destructuring and spread/rest make code concise — use them in assignments and parameters
4. The DOM is your interface to the page — select, modify, create, and remove elements with JavaScript
5. Use try/catch for error handling and debugger/console for troubleshooting
6. async/await is the modern way to handle asynchronous operations — always pair with try/catch

